Design and Empirical Evaluation of an Event-Driven Outbox and CQRS Architecture for High-Concurrency Courier Logistics

Võ Văn Tú Tài, Trần Tấn Tài, and Huỳnh Nam
Faculty of Information System, Industrial University of Ho Chi Minh City
22655911.tai@student.iuh.edu.vn, 22727771.tai@student.iuh.edu.vn

***Abstract*—** High-concurrency courier platforms must ingest barcode scans without relational lock stalls or dual-write loss. We evaluate the live Nexus Event-Driven Outbox and Command Query Responsibility Segregation (CQRS) platform across $R = 5$ runs on NestJS 10 PM2 services, isolated PostgreSQL 16 schemas, and RabbitMQ 3.13 against unsharded (`Sync_ALL`), 16-shard (`Sync_Sharded`), deferred-KPI (`Sync_DeferredKPI`), and SQL Outbox baselines. Across $c \in \{1..100\}$, PostgreSQL lock profiling shows `Sync_ALL` stalls at 753.0 ± 58.1 req/s ($c = 25$, 97.7% lock waits), whereas lock-free SQL Outbox isolation (`pool=21`, 0.0% lock waits) lifts throughput +216.2% over `Sync_ALL` (2,380.8 ± 92.9 req/s, −63.9% p_{95}, $p = 3.0 \times 10^{-7}$) and +56.2% over `Sync_Sharded` (91.8% lock waits). Conversely, component ablation exposes why live Nexus ingress (681.5 ± 32.7 req/s) trails at $c = 1$ (−73.2%) and loses that headroom to `pool=3` limits (−38.6%) plus HTTP lock RPCs and ORM serialization (−53.4%). Under $c = 50$ writers, CQRS reads maintain 0.0% lock waits and cut p_{95} latency −29.7% versus `Sync_ALL` (30.10 vs. 42.81 ms, $p = 0.0057$), while outbox dwell drives 99.72% of post-commit tracking lag. Dual-layer idempotency (300/300 DB writes/1,000 calls), 60/60 FSM guards, 60 s `.retry.1m` quarantine (10/10 healed, 10/10 DLQ), and defects `F1`–`F3` are quantified.

***Index Terms*—** Event-Driven Architecture, Transactional Outbox, CQRS, Log-Based CDC, Idempotency, Lock Contention Ablation, RabbitMQ Dead-Letter Exchange.

I. INTRODUCTION

Modern express courier platforms ingest concurrent barcode scans from sorting hubs and handheld Personal Digital Assistants (PDAs) alongside latency-sensitive public tracking reads [1], [5]. Coupling scan ingestion, transition validation, and multi-dimensional KPI counters inside a single shared ACID transaction causes severe row lock contention, blocking worker pools and degrading tail latency.

Decomposing the monolith into bounded-context microservices eliminates single-schema row locks [6], [7], yet introduces four distributed consistency hazards examined on our running platform:

1) **Dual-Write Consistency Anomaly:** Updating a local database and publishing to a broker outside a single atomic flow risks lost events or phantom states on crashes [1].
2) **Outbox Dwell vs. Service Overhead:** Fixed polling intervals (T_{poll}) and HTTP relays dominate convergence lag, while cross-service RPC locks and small connection pools can erode outbox throughput gains.
3) **Retry Storms & Consumer Races:** Duplicate PDA retries and child scan events arriving ahead of parent waybill creation threaten duplicate KPI rollups or premature failures.
4) **Event Reordering & Queue Stalls:** Delayed or out-of-order (OOO) messages risk corrupting terminal shipment states (`DELIVERED`, `CANCELLED`) unless monotonic state guards and dead-letter escalation are enforced [16].

Without heavy two-phase commit (2PC) or Kafka clusters, we evaluate our live NestJS 10 + PostgreSQL 16 + RabbitMQ 3.13 platform across $R = 5$ independent runs per workload: (RQ1) How much do Outbox decoupling and CQRS isolation reduce relational lock contention and tracking tail latency compared with unsharded, sharded, and deferred-KPI synchronous baselines—and which service overheads explain live ingress gaps via component ablation? (RQ2) Which stage dominates convergence lag using non-circular timing probes, and how fast is pipelined AMQP `ConfirmChannel` batching versus HTTP Management relays? (RQ3) How reliably do dual-layer idempotency, 19-state FSM whitelisting, and multi-tier TTL/DLQ recovery preserve data integrity while exposing active codebase flaws (`F1`–`F3`)?

Key contributions grounded in our live implementation include:

- **Lock-Wait Proof & Ablation:** `pg_locks` profiling proves `Sync_ALL` (97.7%) & 16-shard `Sync_Sharded` (91.8%) stall, while pure SQL Outbox lifts throughput +216.2% ($p = 3.0 \times 10^{-7}$) minus live `pool=3` (−38.6%) and RPC/ORM (−53.4%) taxes.
- **CQRS Isolation & Lag Model:** Cuts read p_{95} −29.7% under $c = 50$ writers (0.0% lock waits), validates Eq. (2) non-circularly (508.0 vs. 553.6 ms), and reaches 29.78× speedup with batch-50 AMQP confirms.
- **Integrity & Code Audit (`F1`–`F3`):** Verifies 300/300 DB commits (1,000 calls), 60/60 FSM guards, exact 60 s `x-death` DLQ TTLs (10/10), and exposes three defects (`F1`–`F3`) with patches.

II. RELATED WORK

Distributed Data & Microservices: The CAP [3] and PACELC [2] theorems formalize latency-consistency tradeoffs underlying BASE workflows [4]. Empirical analysis by Laigner et al. [5] shows database-per-service decomposition [6], [7] shifts consistency guarantees from SQL engines to messaging handlers, necessitating rigorous verification of idempotent consumers and failure semantics.

Transactional Outbox, Polling vs. Log-Based CDC: Hohpe and Woolf [8] and Richardson [1] define the Transactional Outbox with Polling Publishers to prevent dual-write loss without XA locks [9], [15]. Alternatively, log-based Change Data Capture (CDC) tools such as Debezium [18] tail PostgreSQL Write-Ahead Log (WAL) logical replication slots directly, eliminating $T_{\text{poll}}/2$ wait and `SELECT` polling overhead. However, WAL CDC requires managing distributed Kafka Connect pipelines and guarding against PostgreSQL replication-slot WAL disk bloat if connectors stall [18]. Our implementation pairs a lightweight Polling Publisher with RabbitMQ [12] topic routing and Dead-Letter Exchange (DLX) retries, while contrasting latency boundaries against WAL CDC.

CQRS, Idempotency & Event Ordering: CQRS [13], [17] separates write commands from read projections under eventual consistency. Helland [14] motivates idempotent receivers via unique operation keys, while Lamport [16] formalizes logical event ordering and clock skew across asynchronous nodes—directly relevant to preventing OOO projection overwrites without misusing logical clocks as human workflows. Unlike simulator-only evaluations, we benchmark live cluster telemetry and expose concrete service bottlenecks (`F1`–`F3`).

III. SYSTEM ARCHITECTURE AND CORE MECHANISMS

A. Bounded-Context Topology and Service Separation

As illustrated in Fig. 1, the live platform executes natively via PM2 Node.js workers across four isolated PostgreSQL 16 domain schemas (`scan_db`, `shipment_db`, `tracking_db`, `reporting_db`, alongside companion dispatch and payment services): `ScanService` (`scan_db`, `connection_limit = 3`) owns scan ingestion and `CurrentLocation`; `ShipmentService` (`shipment_db`, pool = 15) holds authoritative `Shipment` state; `TrackingService` (`tracking_db`, pool = 3) serves public reads (`TimelineEvent`, `TrackingCurrent`); and `ReportingService` (`reporting_db`, pool = 3) aggregates 8-way `KpiDaily`/`KpiMonthly` rollups (`AggregationJob`). Redis in the gateway BFF handles exclusively WebSocket chat pub/sub. Persistent domain events (`delivery_mode = 2`) route through RabbitMQ's `domain.events` topic exchange.

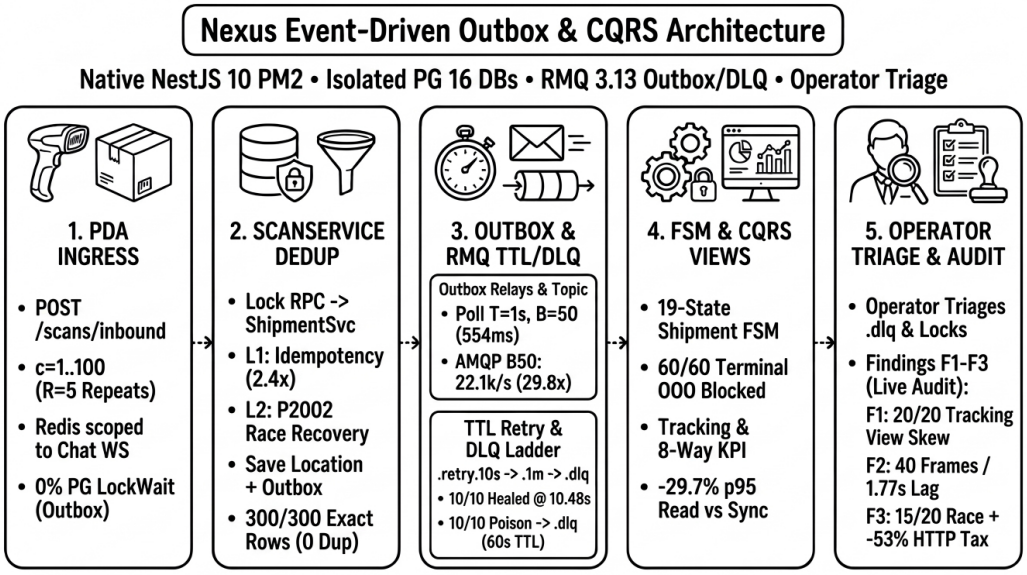


Fig. 1. Live Nexus Event-Driven Outbox and CQRS pipeline across isolated PostgreSQL 16 schemas and RabbitMQ 3.13 (`R = 5` verified metrics), spanning PDA ingress, dual-layer deduplication, outbox relays, 19-state FSM governance, CQRS views, multi-tier TTL/DLQ recovery, and active codebase audit findings (`F1`–`F3`).

B. Dual-Layer Idempotency and Persistent Outbox Writes

Before persisting an inbound scan (`POST /scans/inbound`), `ScanService` executes a synchronous HTTP lock probe (`GET /shipments/:code`) against `ShipmentService`, rejecting locked parcels (HTTP 400) unless authorized by an audit token (`INVENTORY_CHECK`) or active reverse transit (`RETURN_STARTED`). Permitted commands scope keys as `flow:idempotencyKey` through two deduplication layers:

1) **Layer 1 — Sequential Cache Lookup:** Queries `IdempotencyRecord` by scoped key, returning the cached response immediately (4.10 ms pooled median; 4.88 ± 2.87 ms across $R = 5$ runs) on sequential PDA retries.
2) **Layer 2 — Parallel Race Recovery (`P2002`):** Concurrent duplicates bypassing Layer 1 collide on PostgreSQL's unique constraint over `ScanEvent.idempotencyKey` (Prisma error `P2002`), triggering clean read-back of the winner's committed record.
3) **Outbox Append & Audit:** `ScanService` updates `CurrentLocation`, writes a `PENDING` `OutboxEvent` (polled at T_{poll} = 1,000 ms, $B = 50$), saves `IdempotencyRecord`, and logs an audit entry via 7 sequential Prisma operations (atomicity hardening noted under `F3`).

C. Outbox Relays and Propagation Lag Formulation

Our implementation contrasts two outbox relays flushing up to $B = 50$ rows every T_{poll} = 1,000 ms: `ScanService` publishes sequentially via RabbitMQ's HTTP Management `/publish` endpoint, whereas `ShipmentService` maintains a persistent AMQP `ConfirmChannel` [12]. End-to-end client-to-projection visibility decomposes additively:

$$L_{\text{E2E}} = L_{\text{ingest}} + L_{\text{dwell}} + L_{\text{broker}} + L_{\text{cons}} \tag{1}$$

where L_{ingest} is HTTP ingest commit latency, L_{dwell} is outbox queue wait before broker publish confirmation, L_{broker} is broker-to-consumer delivery latency, and L_{cons} is consumer DB commit time (post-commit tracking convergence excludes L_{ingest}). For asynchronous arrivals across periodic T_{poll} ticks, expected dwell is

$$L_{\text{dwell}} \approx \frac{T_{\text{poll}}}{2} + L_{\text{batch}} \tag{2}$$

where $L_{\text{batch}} = \delta_{\text{timer}} + t_{\text{select}} + \frac{\overline{k}+1}{2}(t_{\text{pub}} + t_{\text{mark}})$ captures timer overshoot, SQL `SELECT`, and intra-batch publish/update time measured *independently* from E2E dwell. Under concurrent bursts ($N_0 > B$), drain time quantizes across polling intervals:

$$T_{\text{drain}}(N_0) \approx \left\lceil \frac{N_0}{B} \right\rceil T_{\text{poll}} \tag{3}$$

predicting 4 ticks (4.00 s) for bursts of $N_0 = 200$ events at $B = 50$.

D. FSM Whitelist, CQRS Views, and TTL/DLQ Escalation

Downstream, `ShipmentService` enforces a 19-state finite-state machine (`VALID_TRANSITIONS`) via AMQP push (`noAck: false`), permitting only self-idempotent replays once shipments enter absorbing states (`DELIVERED`, `RETURN_COMPLETED`, `CANCELLED`). `TrackingService` pushes into `TimelineEvent` (`P2002` on `eventId`) and upserts `TrackingCurrent`/`TrackingIndex`, whereas `ReportingService` polls via HTTP (`/get`, T = 1,000 ms, B = 20, `ack_requeue_false`) into `AggregationJob`-deduplicated 8-way KPI tables. Consumer faults ACK the main queue immediately and route via `x-retry-count` across `.retry.10s` (10 s TTL), `.retry.1m` (60 s TTL), and permanent `.dlq` quarantine (retry ≥ 2) for operator triage.

IV. EXPERIMENTAL SETUP AND EMPIRICAL RESULTS

A. Testbed, Stronger Baselines, and Ablation Design

All benchmarks executed against the live cluster on an Apple M4 host (10 cores, 16 GB RAM, macOS 15.8, Node.js v26.0.0) backed by PostgreSQL 16.15 (`max_connections = 100`) and RabbitMQ 3.13.7 (`rates_mode = basic`). Before timed measurements, a 1,995 ms warmup (40 writes + 60 reads/target) warmed HTTP sockets and Prisma engines; pre-benchmark SQL dumps were restored afterward. Every latency/scaling experiment executed R = 5 independent block-rotated repetitions, sampling `pg_stat_activity`, `pg_locks WHERE NOT granted`, and `os.cpus()` busy time at 5 ms resolution. Statistical tests compare n = 5 independent run summaries (paired t-tests, df = 4; exact two-sided Wilcoxon `min p = 0.0625` at n = 5) alongside exact Clopper–Pearson 95% binomial intervals, eliminating intra-run pseudoreplication.

TABLE I
LIVE COMPARATORS, POOL SETTINGS, AND R = 5 SYNTHETIC WORKLOADS

Suite / Target	Config & Pool	Workload (R = 5) & Measurement
`Nexus_EDA` (`:3006`)	**PM2 NestJS/Prisma; pool = 3 + HTTP Lock RPC**	**Exp 1: $c \in \{1, 10, 25, 50, 100\}$ × 200 × 5 runs; Exp 1C: 25R vs Idle & c=50W (200 × 5 reads)**
`Ablation_Outbox_P3`	Raw PG Outbox Worker; pool = 3 (no HTTP/Prisma)	Isolates `connection_limit=3` PM2 pool cap vs `Ablation_Outbox_SQL` & Prisma/HTTP tax
`Ablation_Outbox_SQL`	**Raw PG Outbox Worker; pool = 21 (local lock/tx)**	**Isolates pure Single-TX SQL Outbox pattern at matched PostgreSQL pool = 21**
`Sync_ALL` / `Sync_Sharded` / `Sync_DeferredKPI`	Atomic SQL TX (`:13039`, pool = 21, `LIMIT 10` read)	1-row `ALL`|`ALL`|`ALL` KPI vs 16-shard hashed KPI vs KPI-free write/read baselines
`LockRPC_Probe` (`:3002`)	Live Shipment GET RPC (pool = 15)	Isolates synchronous HTTP lock check cost; 100 reqs/tier × 5 runs
Exp 2A–2D / 3 / 4A–4B	Isolated cluster sweep; clean queues & *x-death*	2A: 200 staggered; 2B/C: 5 × 200; 2D: independent micro-probes; 3: 1,000 HTTP + 150 dup; 4A/B faults

Notes: Warmup 1,995 ms; `max_connections=100`; 5 ms `pg_locks` & `os.cpus()` polling.

B. Scaling, Component Ablation, and Lock Proof (RQ1)

Table II and Fig. 2(a) expose three structural findings across 100.0% successful (`2xx`) requests:

1) Direct Lock-Contention Proof: At c = 25, `Sync_ALL` stalls at 753.0 ± 58.1 req/s while PostgreSQL sampling captures 97.7% lock-wait intervals (3,706 `txid` waits; up to 20 of 21 connections blocked) leaving host CPU under-utilized at 49.6%. Contrary to naive intuition, even 16-shard `Sync_Sharded` still suffers 91.8–94.7% lock-wait sampling (470–543 `txid` waits, max 9–12 blocked) across c = 25..100 because 8-way KPI groupings hash-collide across concurrent updates, capping throughput near ~1,524 req/s (76.7% CPU). Removing synchronous KPI updates (`Sync_DeferredKPI`) eliminates lock waits (0–2.2%) and scales to 2,926.4 ± 214.6 req/s (c = 50).

2) Pure Outbox Gains & Honest Ablation Tax: Matched at pool = 21, `Ablation_Outbox_SQL` maintains 0.0% lock waits (0 `txid` waits) and reaches 2,380.8 ± 92.9 req/s (c = 25, p_{95} = 29.66 ms), outperforming `Sync_ALL` by +216.2% (3.16×, −63.9% p_{95}, $p = 2.99 \times 10^{-7}$) and `Sync_Sharded` by +56.2% ($p = 3.76 \times 10^{-5}$). Conversely, live `Nexus_EDA` peaks at only 698.2 ± 20.5 req/s (c = 100), trailing `Sync_ALL` at c = 1 (146.0 vs. 545.0 req/s, −73.2%, $p = 3.78 \times 10^{-5}$) and tying `Sync_ALL` at c = 25 (681.5 vs. 753.0 req/s, p = 0.120 n.s.; p_{95} −7.3%, p = 0.044) while trailing `Sync_Sharded` (−55.3%). Two-step ablation attributes this loss cleanly: (a) capping pool size at 3 (`Ablation_Outbox_P3`) cuts SQL Outbox throughput −38.6% (2,380.8 → 1,461.6 req/s, $p = 9.38 \times 10^{-5}$), and (b) adding cross-service HTTP lock checks (`LockRPC_Probe`: 2,703.7 ± 315.3 req/s at c = 25) plus 7 Prisma ORM calls imposes an additional −53.4% ingress penalty (1,461.6 → 681.5 req/s, $p = 1.12 \times 10^{-5}$).

3) CQRS Read Isolation Under c = 50 Writes (Exp 1C): As shown in Table II and Fig. 2(b), under Idle load `Sync` bounded `LIMIT 10` queries (p_{95} = 6.31–7.80 ms) run much faster than `Nexus_CQRS_Read` (p_{95} = 40.04 ± 6.08 ms; p_{50} = 18.89 ms), which scans unbounded `TimelineEvent` histories on `pool=3`. However, under c = 50 writers, `Sync_ALL` (100.0% read-period lock waits; max 20 blocked) jumps +35.94 ± 0.60 ms in median latency ($p = 1.89 \times 10^{-8}$) to p_{95} = 42.81 ± 2.92 ms, and `Sync_Sharded` (100.0% lock waits) jumps +19.70 ms in median (+22.00 ms p_{95}) to 28.31 ± 5.26 ms. Meanwhile `Nexus_CQRS_Read` sustains 0.0% lock waits—shifting +5.51 ± 2.19 ms in median (p = 0.0049) purely from shared-host CPU saturation (92.3%)—cutting p_{95} tail read latency −29.7% versus `Sync_ALL` (30.10 vs. 42.81 ms, p = 0.0057; −37.0% mean across 5/5 runs) and matching `Sync_Sharded` (p = 0.505 n.s.).

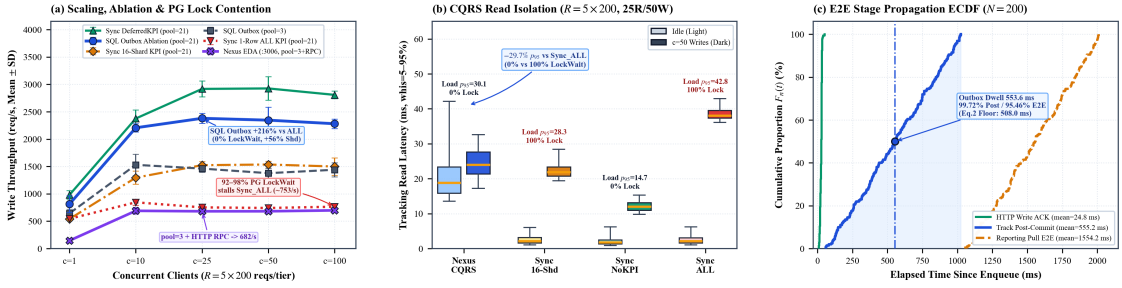


Fig. 2. Genuine R = 5 cluster telemetry: (a) Write scaling & 4-step ablation with PostgreSQL lock-wait breakdown (Mean ± SD); (b) CQRS read isolation under 25 readers vs. Idle and c = 50 concurrent writers (5th–95th whiskers, N = 1,000 queries/box); and (c) Post-commit stage propagation ECDF (N = 200 joined scans).

TABLE II
R = 5 SCALING, COMPONENT ABLATION, PG LOCK/CPU TELEMETRY (EXP 1) & CQRS READ ISOLATION (EXP 1C)

Experiment / Architecture	Tier / Cond.	req/s or Read p_{50}	p_{95} (ms) Mean ± SD	PG Lock% (maxBlk)	Host CPU% / Key Finding
`Nexus_EDA (:3006)`	Write c=1	146.0 ± 30.0/s	18.45 ± 2.77	0.0% (0)	40.3% (−73% Sync)
`Nexus_EDA (:3006)`	Write c=25	681.5 ± 32.7/s	76.23 ± 5.62	0.0% (0)	65.0% (−53% RPC/ORM)
`Ablation_Outbox_P3`	Write c=25	1,461.6 ± 65.5/s	38.63 ± 4.75	0.0% (0)	55.2% (−39% pool=3)
`Ablation_Outbox_SQL`	**Write c=25**	**2,380.8 ± 92.9/s**	**29.66 ± 6.39**	**0.0% (0)**	**78.1% (+216% ALL)**
`Sync_DeferredKPI`	Write c=25	2,917.6 ± 144.9/s	17.21 ± 3.43	0.0% (0)	78.0% (KPI-free)
`Sync_Sharded` (16-Shd)	Write c=25	1,524.3 ± 68.9/s	30.27 ± 3.63	91.8% (9)	76.7% (470 txid)
`Sync_ALL` (1-row KPI)	**Write c=25**	**753.0 ± 58.1/s**	**82.20 ± 3.57**	**97.7% (20)**	**49.6% (3,706 txid)**
`Nexus_EDA / Sync_ALL`	Write c=100	698.2 / 765.3/s	187.8 / 159.7	0.0 / 97.7%	66.9% / 47.7% CPU
`Nexus_CQRS_Read`	**Idle → c=50**	**18.89 → 24.40 ms**	**40.04 → 30.10**	**0.0 → 0.0%**	**92.3% (−29.7% ALL)**
`Sync_Sharded Read`	Idle → c=50	2.12 → 21.82 ms	6.31 → 28.31	0.0 → 100%	83.9% (maxBlk=11)
`Sync_DeferredKPI Read`	Idle → c=50	1.83 → 12.00 ms	7.80 → 14.69	0.0 → 0.0%	85.4% (`LIMIT 10`)
`Sync_ALL Read`	Idle → c=50	2.25 → 38.19 ms	7.76 → 42.81	0.0 → 100%	49.5% (+35.9ms p50)

Notes: Paired `Ablation_Outbox_SQL` vs. `Sync_ALL` $p = 2.99 \times 10^{-7}$; vs. `Sync_Sharded` $p = 3.76 \times 10^{-5}$. Read $c = 50$ `Nexus_CQRS` vs. `Sync_ALL`: −29.7% p_{95} ($p = 0.0057$).

C. Propagation Lag, Non-Circular Eq. (2), and Transport (RQ2)

Across $R = 5 \times 40 = 200$ clean staggered scans (Exp 2A, Table III, Fig. 2c), HTTP ingest finishes in 24.76 ms (`run SD` 0.78 ms; $p_{50} = 26.92$ ms). Outbox polling dwell averages 553.62 ms (`run SD` 14.30 ms; pooled 95% CI [515.6, 592.2] ms; $p_{95} = 981.0$ ms)—contributing **99.72% of post-commit tracking convergence** (553.62 / 555.16 ms; $R^2 = 99.99\%$) and **95.46% of full client E2E lag** Eq. (1) (553.62 / 579.92 ms). AMQP push adds only 1.55 ms to `TrackingService` and 0.50 ms to `ShipmentService` (−6..0 ms timestamp ordering skew arises because `ScanService` calls HTTP `publishEvent()` immediately before SQL `markPublished()`), whereas `ReportingService`'s 1,000 ms HTTP `/get` pull phase-locks at 1,000.60 ms post-publish (1,554.21 ms post-commit).

Non-Circular Eq. (2) Evaluation (Exp 2D): Independent component probes measured outside Exp 2A give $\delta_{\text{timer}} = 0.945$ ms ($n = 15$), $t_{\text{select}} = 1.215$ ms ($n = 50$), $t_{\text{mark}} = 0.245$ ms ($n = 100$), and HTTP publish $t_{\text{pub}} = 1.357$ ms (Exp 2C, $n = 1{,}000$). With Exp 2A's 159.6 ms mean inter-arrival interval ($\bar{k} = 6.27$ rows/tick), Eq. (2) predicts raw-SQL $L_{\text{batch}} = 7.98$ ms and a lower-bound dwell floor of 507.98 ms—leaving a transparent +45.6 ms margin relative to measured 553.62 ms attributable to Prisma query-engine wrappers and sequential HTTP connection teardown in the relay worker (within ~2.3× Monte-Carlo SE ≈ 19.8 ms).

Under $R = 5$ bursts of 200 writes ($c = 50$, Exp 2B, Fig. 3a), outbox draining finishes in 4,060.2 ± 299.7 ms (DB span 3,941.0 ± 324.4 ms), tracking Eq. (3)'s 4,000 ms prediction. In Exp 2C ($R = 5 \times 200$ messages), switching HTTP Management sequential publishing (271.6 ± 36.0 ms; 745.8 msg/s) to native AMQP `ConfirmChannel` cuts run totals to 113.1 ± 6.5 ms sequentially (2.40 ± 0.19×) and **9.15 ± 1.06 ms** when pipelined in batches of 50 (22,074 ± 2,407 msg/s; **29.78 ± 3.31× speedup**).

D. Idempotency, FSM Safety, Locks, and TTL/DLQ Proof (RQ3)

Across $R = 5$ clean runs covering 1,000 HTTP calls over 300 unique keys (Exp 3, Fig. 3b), dual-layer idempotency returned 1,000/1,000 (100.0%) `2xx` responses while creating strictly 300/300 `ScanEvent`, 300/300 `OutboxEvent`, and 300/300 `AggregationJob` records (60/60 per run). Layer-1 cache replay sped up sequential retries 2.42 ± 0.48× across runs (run medians 4.88 vs. 11.51 ms Fresh; paired t-test $p = 0.0164$; 5/5 runs faster), while Layer-2 `P2002` resolved all 6× parallel storms cleanly (55.33 ms median). Injecting 150 duplicate RabbitMQ envelopes (5 × 30) leaked 0/150 tracking rows and 0/150 KPI increments. Against 60 OOO/terminal AMQP attacks (Exp 4A), the 19-state FSM preserved 60/60 (100.0%; Clopper–Pearson 95% CI [94.04%, 100.0%]) authoritative states and 60/60 phased lock probes (30 blocked `400`, 30 overridden `201`).

In Exp 4B (Fig. 3c, $N = 20$ faults), 10/10 (100.0%; 95% CI [69.15%, 100.0%]) orphan events healed automatically at 10.48 s after delayed parent creation at 3.11 s, and 10/10 ([69.15%, 100.0%]) permanent poison tokens reached `.dlq` (20/20 conservation, [83.16%, 100.0%]). Crucially, Management API queue polling (`/api/queues` under `rates_mode = basic`) exhibits a ~4–5 s stats-collection lag (`.retry.1m` observed 14.60 → 74.45 s), whereas extracting raw AMQP `x-death` headers directly from `.dlq` messages verifies an **exact 60 s `.retry.1m` TTL** (`1794324717 - 1794324657 = 60 s`, 70 s total broker TTL across 10/10 frames).

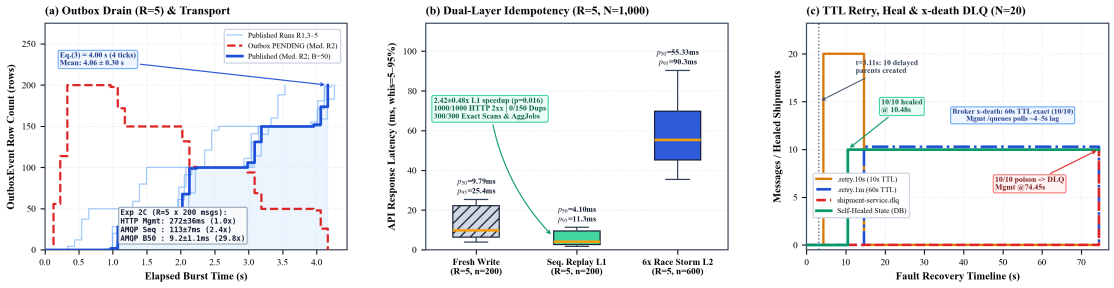


Fig. 3. Burst drain, deduplication, and DLQ telemetry: (a) R = 5 burst-200 drain staircases (T_{poll} = 1,000 ms, B = 50) + Exp 2C transport inset; (b) Dual-layer idempotency latency across 1,000 HTTP calls (R = 5); and (c) Live TTL retry recovery (`10/10` healed @ 10.48 s) and broker `x-death`-verified 60 s `.retry.1m` quarantine (`10/10` → `.dlq`).

TABLE III
LAG DECOMPOSITION (EXP 2A/2D), TRANSPORT (EXP 2C), AND RELIABILITY/AUDIT SUMMARY (EXP 2B–4B)

Pipeline / Verification Target	Sample (R=5)	Mean / Outcome	SD / CI$_{95}$ / Rate	Share / Key Finding
2A: Ingest → Outbox Dwell	**N=200 (5×40)**	**24.76 → 553.62 ms**	**SD 14.30 (run) p_{95} = 981 ms**	**99.72% Post / 95.46% E2E**
2A: AMQP Push / Rep. Pull	N=200 (5×40)	1.55 / 1,000.6 ms	555.2 / 1,554 ms	FSM 0.50 ms; 1 s phase-lock
2D: Non-Circular Eq. (2)	n=15..1,000	Floor 507.98 ms	L_{batch} = 7.98 ms	+45.6 ms ORM relay margin
2B: Burst-200 Drain Curve	5 × 200 writes	4,060.2 ms	SD 299.7 (DB 3,941)	Matches Eq. (3) 4,000 ms
2C: HTTP / AMQP Seq / B50	**5 × 200 msgs**	**271.6 / 113.1 / 9.15**	**0.75k / 1.77k / 22.1k/s**	**2.40× Seq / 29.78× Batch50**
3/3C: Dual-Layer Idem. & F2	1,000 + 150 dup	300/300 Exact; 0 dup	2.42× L1 (p=0.016)	F2: 40 popped / 1.77 s tail
4A: FSM OOO & Lock Probes	60 + 60 + 20	60/60 FSM; 60/60 Ph.	[94.04%, 100.0%]	F1: 20/20 Skew; F3: 15/20 Race
4B: Retry Heal & `.dlq` TTL	**10 + 10 faults**	**10/10 Heal; 10/10 DLQ**	**[69.15%, 100%]; [83.2, 100%]**	**60 s exact AMQP `x-death` TTL**

Notes: Post-commit convergence excludes HTTP ingest (24.76 ms). AMQP −6..0 ms skew stems from publishing ahead of SQL `markPublished()`. `.dlq` verify 60 s `x-death` TTL.

V. DISCUSSION AND ACTIVE CODEBASE AUDIT (F1–F3)

Synthesizing RQ1–RQ3: Live lock profiling confirms that single-row (`Sync_ALL`) and even 16-shard (`Sync_Sharded`) synchronous KPI counters trigger severe PostgreSQL `txid` lock contention (92–98% lock-wait sampling). Outbox isolation completely eliminates write-side lock contention (0.0%), unlocking +216.2% speedup in raw SQL (`Ablation_Outbox_SQL`). Yet real microservice boundaries must earn that headroom: running `ScanService` on `connection_limit=3` alongside cross-service HTTP Lock RPCs and ORM serialization drops ingress −38.6% then −53.4%, showing why local lock replication and pooled connections are prerequisite to realizing Outbox throughput gains. Likewise, CQRS shields tracking queries from 100% `Sync_ALL` write-lock stalls (−29.7% p_{95}), yet bounded `LIMIT` projections remain essential so idle reads match SQL speed. Finally, while T_{poll} = 1,000 ms explains 99.72% of post-commit lag, sub-50 ms SLOs warrant PostgreSQL `LISTEN/NOTIFY` or Debezium WAL CDC [18] (trading polling wait for replication-slot management).

Three Unpatched Cluster Findings (F1–F3): Fault instrumentation exposed three active defects in the evaluated binaries with concrete remediations:

1) **F1 — CQRS Read View Skew (20/20 Shipments):** While `ShipmentService` rejected 60/60 OOO terminal attacks, missing-snapshot OOO envelopes cause `TrackingService` to infer status directly from the late event type, overwriting `TrackingCurrent` across all 20/20 test parcels. *Fix:* Enforce monotonic `stateVersion` or `occurredAt` guards on projection writes using authoritative state events.
2) **F2 — HTTP Pull Crash Exposure (40 Frames / 1.77 s Tail):** `ReportingService` pulls 20-message batches via HTTP `ack_requeue_false` before `AggregationJob` commits complete; our live 40-frame probe showed all 40/40 messages popped prematurely ahead of a 1,771 ms DB commit tail (at-most-once crash gap). *Fix:* Migrate to AMQP push with manual `basicAck` executed strictly post-commit.
3) **F3 — Async Lock TOCTOU & Non-Atomic Outbox (15/20 Race + −53.4% Tax):** Interleaved `INVENTORY_CHECK` bursts triggered an asynchronous unlock event that cleared `isLocked = false` mid-tick, allowing 5/10 subsequent locked scans to pass (`15/20` overall). Moreover, sequential HTTP Lock RPCs plus unbundled `ScanEvent` and `OutboxEvent` writes halve ingress throughput. *Fix:* Replicate lock flags asynchronously into `scan_db` and wrap local scan + outbox writes in `prisma.$transaction`.

VI. CONCLUSION, LIMITATIONS, AND REPRODUCIBILITY

Conclusion: Across R = 5 repeated live-cluster runs paired with PostgreSQL lock telemetry and component ablation, we demonstrate that lock-free SQL Outbox isolation raises write throughput +216.2% over unsharded `Sync_ALL` and +56.2% over 16-shard `Sync_Sharded` (`0.0%` vs `91.8–97.7%` lock waits), while pinpointing how `pool=3` limits (−38.6%) and synchronous HTTP Lock RPCs + ORM costs (−53.4%) bottleneck the deployed service (`F3`). Under c = 50 writers, CQRS read isolation eliminates 100% write-lock bleed (−29.7% p_{95} vs. `Sync_ALL`), non-circular Eq. (2) bounds outbox dwell (99.72% post-commit share), batch-50 AMQP confirms reach 29.78× speedup, dual-layer idempotency holds 300/300 exact commits (1,000 calls), and broker `x-death` headers confirm exact 60 s retry-queue TTL quarantine alongside code findings `F1`–`F3`.

Limitations & Reproducibility: All tests ran on a single Apple M4 host (native PM2 services + Docker PG/RabbitMQ, excluding WAN latency) using synthetic 200–1,000 call bursts and remain intentionally pre-patch for `F1`–`F3`. Harness scripts (`run_benchmarks.mjs`, `baseline_worker.mjs`, `analyze_benchmarks.py`, `generate_distributed_charts.py`) and raw R = 5 CSV/JSON traces are archived at `https://github.com/VoVanTuTai/logistics-management-system`.

REFERENCES

[1] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018 (520 pp., ISBN 978-1-61729-454-9).
[2] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012.
[3] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002.
[4] D. Pritchett, "BASE: An Acid alternative," *ACM Queue*, vol. 6, no. 3, pp. 48–55, May 2008.
[5] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3348–3361, 2021.
[6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021 (616 pp., ISBN 978-1-4920-3401-8).
[7] V. Vernon, *Implementing Domain-Driven Design*. Upper Saddle River, NJ, USA: Addison-Wesley, 2013 (656 pp., ISBN 978-0-321-83457-7).
[8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, USA: Addison-Wesley, 2003 (ISBN 978-0-321-20068-6).
[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017 (616 pp., ISBN 978-1-4493-7332-0).
[10] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, 2011, pp. 1–7.
[11] J. Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction," *LinkedIn Engineering Blog*, Dec. 2013.
[12] A. Videla and J. J. W. Williams, *RabbitMQ in Action: Distributed Messaging for Everyone*. Shelter Island, NY, USA: Manning Publications, 2012 (312 pp., ISBN 978-1-935182-97-9).
[13] G. Young, "CQRS Documents," *CQRS Info Technical Notes*, Nov. 2010.
[14] P. Helland, "Idempotence is not a medical condition," *ACM Queue*, vol. 10, no. 4, pp. 30–46, Apr. 2012.
[15] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, San Francisco, CA, USA, 1987, pp. 249–259.
[16] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.
[17] M. Fowler, "CQRS," *MartinFowler.com Architecture Series*, Jul. 2011.
[18] G. Morling, "Reliable microservices data exchange with the Outbox pattern," *Debezium Engineering Blog*, Feb. 2019.